

Important notes before working on this assignment:

- It is highly recommended that you check **the material (available on D2L)** before working on this assignment. On D2L, you will find:
 - **Multithreading Java**: examples showing how to create, terminate, stop, and interrupt threads
 - **Synchronization Java**: examples showing how to create Mutex Lock and Semaphore shared between cooperating threads
- All work submitted for this assignment must reflect your own effort and understanding. The use of ANY generative AI tools for completing this assignment is **strictly prohibited**. Such use will be considered academic misconduct. Students found in violation of this policy will receive an 'F' for the work in question and may face further academic discipline according to the NEIU Student Code of Conduct.
- From time to time, students may be required **to attend a mandatory meeting with the instructor**, so that the instructor can assess the student's understanding of course content. If a student is suspected of or reported for violating the academic integrity policy above, the instructor will consult with the Dean of Students.
- **By submitting this assignment, you confirm that this work is entirely your own. You have not used unauthorized external help or generative AI tools (unless specifically allowed in the instructions). You have only utilized resources provided by the instructor, TA, or course tutors.**

Mutex locks and semaphores, as discussed in class, are different techniques to solve the race condition and to ensure efficient synchronization between cooperating threads and processes. In this assignment, you will create multithreaded programs and use locks and semaphores to solve synchronization problems between the cooperating threads. As mentioned in the second set of Java sketches:

- Mutex Lock, in literature, uses acquire() and release(). However, in the standard library of Java, these functions are lock() and unlock(), respectively. *The same functionalities but with different names.*
- Semaphore, in literature, uses wait() and signal(). However, in the standard library of Java, these functions are acquire() and release(), respectively. *The same functionalities but with different names.*

In this assignment, you can develop the required tasks on the main OS of your computer or the Ubuntu VM you installed, and you can use any Java IDE you find appropriate (e.g., NetBeans, Eclipse, IntelliJ). ***You may also use Python instead of Java.***

Question 1 (5 points): Provided with the assignment is a Java program that -in the main method- creates two threads that share the balance of a certain bank account and work in parallel. As shown in the main function:

- The first thread works in an infinite loop and deposits random amounts to the shared balance.
- The second thread works in an infinite loop and withdraws random amounts from the shared balance.

Use *mutex lock(s)* to implement the synchronization between such competing threads. The comments in the Java files explain the required functionalities and guide you to ***where exactly to place your implementation***, read them carefully before starting coding. **DO NOT CHANGE** any existing code (e.g., function name, variable name) unless you use Python (in that case you will build similar existing code).

Note: the threads running in the above questions (questions 1 and 2) are implemented to run in an infinite loop, so at some point, you may need to force-stop the program.

Question 2 (5 points): Provided with the assignment is a Java program that -in the main method- creates two threads that share an array of integer values (with a size of N) and work in parallel. As shown in the main function:

- The first thread works in an infinite loop, producing random integer values to be added to the shared array only if there is a free space in the shared array.
- The second thread works in an infinite loop, consuming values from the shared array (overwriting their values with -1) only if there is an available item in the buffer.

Use *semaphore(s)* to implement the synchronization. The comments in the Java files explain the required functionalities and guide you to ***where exactly to place your implementation***, read them carefully before starting coding. **DO NOT CHANGE** any existing code (e.g., function name, variable name) unless you use Python (in that case you will build similar existing code).

Note: the threads running in the above questions (questions 1 and 2) are implemented to run in an infinite loop, so at some point, you may need to force-stop the program.

Question 3 (5 points): Provided with the assignment are 3 input files (Input1, Input2, and Input3). You will also create one output file (initially empty) yourself. Each of the 3 input files has 10,000 employee data records. Each record (line) has an employee's ID, full name, title, and salary. ****Note: these values were randomly generated.***

You are asked to calculate some statistics about the **salary field only**, and the calculated statistics are then written to the output file. In order to maximize the speed, the main thread of your Java program will create 3 separate threads to work in parallel - each thread will:

1. First read one of the input files (thread1 reads Input1, thread2 reads Input2, thread3 reads Input3).
2. Process the input data record (line) to get the salary value, and calculate the required statistics.
3. Append the calculated statistics in the output file, using the following format:

Thread Name: *print the name of the thread (e.g., thread 1)*
File Name: *print the name of the file the thread used (e.g., Input1.txt)*
The Maximum Salary: *print the maximum value (e.g., 100000)*
The Minimum Salary: *print the minimum value (e.g., 60000)*
The Average Salary: *print the average value (e.g., 80000)*
The task was concluded: *print the current system time (the local date and time of your computer).*

For multiple threads to access the output file (shared resource), you need to use either *mutex lock(s)* or *semaphore(s)* to implement such synchronization. The main thread should wait for all threads to finish their work (you can use Join for this purpose) and then append the following string to the output file:

This summary was completed using three parallel threads.
*This work was done by: your first and last name, and your
NEIU ID number*

Submission:

1. This is an **individual assignment** -- Cheating/plagiarism will be checked and violators will receive a zero.
2. Your submission to the folder titled Assignment 2 under the D2L Assignments tab should contain one file, ***Runtime.pdf***:
 - a. For each question/part, copy the **full** Java (or Python) code and add several screenshots of the **output after running the code**.
 - b. For both Q1 and Q2: Pick one screenshot of the output after running the code and explain (using your own words) how the synchronization between the two threads resulted in the output displayed.
 - c. For Q3, identify the critical section(s) of your code? And explain why?
3. Check the due date on D2L, make sure you submit your work early enough before the due date.
 - You can submit your assignment within **24-hours after the due date** to be graded for a maximum of **70%** of the assignment's grade.
 - After the 24 hour period, ***your late submission will not be accepted.***
 - Work on the required tasks early enough to avoid any technical problems.